

基本交換法（バブルソート）

アルゴリズムの中でも、最もシンプルで理解しやすい並べ替えの手法が「基本交換法（バブルソート）」です。

基本情報技術者試験でも、アルゴリズムの入門として必ずと言っていいほど登場します。

1. バブルソートの名前の由来

「バブル」とは、お風呂の「泡（あわ）」のことです。重いデータが下に沈み、軽いデータが「泡」のようにプクプクと一つずつ上に上がっていく様子に似ていることから、この名前がつけました。

2. アルゴリズムの仕組み

一言で言うと、「隣り合う2つの数字を比べて、逆順だったら入れ替える」という動作を繰り返すだけです。

「小さい順（昇順）」に並べ替えるときの手順を見てみましょう。

- 1. **比較**：左端の要素とその隣の要素を比べる。
- 2. **交換**：左の方が大きければ、場所を入れ替える（右の方が大きければそのまま）。
- 3. **移動**：1つ隣にずれて、また隣同士を比べる。
- 4. **確定**：これを右端まで繰り返すと、一番大きな数字が右端に「確定」します。
- 5. **反復**：確定したものを除いて、また最初から繰り返します。

3. 具体的な例（4, 2, 5, 1 を並べ替える）

[4, 2, 5, 1] という数字を並べ替える方法を考えます。この数列を、縦と横を逆にして、下記のように図示します。**#**の数が数字の大きさを表しています。

Status	比較対象	数値	図
		4	####
		2	##
		5	#####
		1	#

【1周目】

Step 1

[4, 2, 5, 1] → 4と2を比較。

Status	比較対象	数値	図
	*	4	####
	*	2	##
		5	#####
		1	#

*は比較する対象の数字です。

4が大きいので交換 → [2, 4, 5, 1]

Status	比較対象	数値	図
	*	2	##
	*	4	####
		5	#####
		1	#

Step 2

- [2, 4, 5, 1] → 4と5を比較。

Status	比較対象	数値	図
		2	##
	*	4	####
	*	5	#####
		1	#

- 5が大きいのでそのまま → [2, 4, 5, 1]

Status	比較対象	数値	図
		2	##
	*	4	####
	*	5	#####
		1	#

Step 3

- [2, 4, **5**, 1] → 5と1を比較

Status	比較対象	数値	図
		2	##
		4	####
	*	5	#####
	*	1	#

- 5が大きいので交換 → [2, 4, **1**, 5]

Status	比較対象	数値	図
		2	##
		4	####
	*	1	#
	*	5	#####

→ 「5」が最大値として確定！

Status	比較対象	数値	図
		2	##
		4	####
		1	#
確定		5	\$\$\$\$

確定したものは、'\$'で表します。

【2周目】

Step 1

- [2, 4, 1, | 5] → 2と4を比較。

Status	比較対象	数値	図
	*	2	##
	*	4	####
		1	#
確定		5	\$\$\$\$

- 4が大きいのでそのまま → [2, **4**, 1, | 5]

Status	比較対象	数値	図
	*	2	##
	*	4	####
		1	#
確定		5	\$\$\$\$

Step 2

- [2, **4**, 1, | 5] → 4と1を比較。

Status	比較対象	数値	図
		2	##
	*	4	####
	*	1	#
確定		5	\$\$\$\$

- 4が大きいので交換 → [2, **1**, 4, | 5]

Status	比較対象	数値	図
		2	##
	*	1	#
	*	4	####
確定		5	\$\$\$\$

→ 「4」が確定！

Status	比較対象	数値	図
		2	##
		1	#
確定		4	\$\$\$\$
確定		5	\$\$\$\$

【3周目】

Step 1

- [2, 1, | 4, 5] → 2と1を比較。

Status	比較対象	数値	図
	*	2	##
	*	1	#
確定		4	\$\$\$\$
確定		5	\$\$\$\$

- 2が大きいので交換 → [1, 2, | 4, 5]

Status	比較対象	数値	図
	*	1	#
	*	2	##
確定		4	\$\$\$\$
確定		5	\$\$\$\$

→ 「2」が確定！

Status	比較対象	数値	図
		1	#
確定		2	\$\$
確定		4	\$\$\$\$
確定		5	\$\$\$\$

→ 最後の一つ「1」も自動的に確定！

Status	比較対象	数値	図
確定		1	\$
確定		2	\$\$
確定		4	\$\$\$\$
確定		5	\$\$\$\$

4. バブルソートの特徴（メリット・デメリット）

項目	特徴
わかりやすさ	プログラムが非常に単純で、初心者でも理解しやすい。
スピード	遅い。データが増えると、比較回数が一気に増えるため、実務で大量のデータを扱うのに向きません。
計算量	$O(n^2)$ 。データ数が10倍になると、かかる時間は100倍になります。

5. 実装

疑似言語による実装

```

o bubbleSort(整数型の配列: data)
  整数型: i, j, temp, n
  n ← dataの要素数
  for (i を 0 から n - 2 まで 1 ずつ増やす)
    for (j を 0 から n - i - 2 まで 1 ずつ増やす)
      if (data[j] > data[j + 1])
        temp ← data[j]
        data[j] ← data[j + 1]
        data[j + 1] ← temp
      endif
    endfor
  endfor

```

初心者が押さえるべきポイント：

- **2重のループ**: 外側のループは「何個の数字を確定させたか」を数え、内側のループで実際に隣同士を比較していきます。
- **比較範囲の工夫**: 1周回ごとに最大値が右端へ移動して「確定」するため、次にチェックする範囲を1つずつ狭めて無駄な計算を減らしています。
- **入れ替え（Swap）**: 値を上書きして消してしまわないよう、**temp** という「予備の箱」を使って値を一時的に避難させて交換します。

C言語による実装

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;

    // 全体を走査するためのループ
    for (i = 0; i < n - 1; i++) {
        // 隣り合う要素を比較するループ
        // (n-i-1) としているのは、右端から順に確定していくため
        for (j = 0; j < n - i - 1; j++) {
            // 左の要素が右より大きければ入れ替える（交換）
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int data[] = {4, 2, 5, 1}; // 説明で使った例
    int n = 4; // 要素数

    printf("ソート前: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    bubbleSort(data, n);

    printf("ソート後: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    return 0;
}
```